COSC4606 Proposal

Group 1

Onboarding onto Complex Databases with an Explainable Text-to-SQL Assistant

Problem and Motivation:
Enterprise databases often span several schemas and hundreds of tables, with cryptic names and relationships that are poorly documented or never declared. This slows onboarding and leaves new employees depending on a few experts. Large language models (LLMs) can turn plain-language questions into SQL, but current approaches have three weaknesses:

1. Putting every table definition in the prompt is expensive, and accuracy drops when the model has to find a few relevant tables among many.
2. Retrieval-based tools such as Vanna treat each table as a separate text document and ignore foreign keys. They often miss "bridge" tables that are needed for a join but don't look related to the question.
3. Most systems don't explain how a query was built, so a new hire learns nothing about the database and can't easily tell when an answer is wrong.

Commercial systems such as Snowflake's Cortex Analyst and WrenAI improve accuracy with semantic models that people have to build and maintain by hand. Research on benchmarks such as Spider (Yu et al., 2018) and BIRD (Li et al., 2023) shows that matching questions to the right tables and columns is a major source of errors on large databases.

Our project copies the database's metadata into a graph database, where following the connections between tables is fast and easy. The system can then look up how tables should be joined instead of guessing, and use that to write better SQL.

Objectives:
1. Design and implement a Graph metadata layer
2. Combine vector retrieval with graph traversal to select the minimal sub-schema needed to answer a question, including bridge tables that retrieval alone misses.
3. Show the user the tables used, the join path with the meaning of each join, and a plain-language walkthrough of the generated SQL, so that every answer teaches something about the database.
4. Measure whether graph guidance improves execution accuracy over flat retrieval, especially on multi-hop, cross-schema questions.

Proposed Approach

1. **Metadata extraction & integration**
   An ingestion component queries PostgreSQL's *information_schema* and *pg_catalog* to extract all relevant metadata including:
   - Schemas, tables, and column names with their data types.
   - Primary keys, foreign keys, and unique constraints.
   - Column comments

A common problem in real systems is a lack of foreign key declaration. To mitigate this problem we will attempt to infer relationships between tables via candidate columns – columns with matching names, compatible data types, and value overlap. This step in the approach is a data integration task: relational metadata is extracted, transformed, and loaded into a graph model. Any changes in schema can be resolved through another ingestion step.

2. Graph Data Modeling in Neo4j

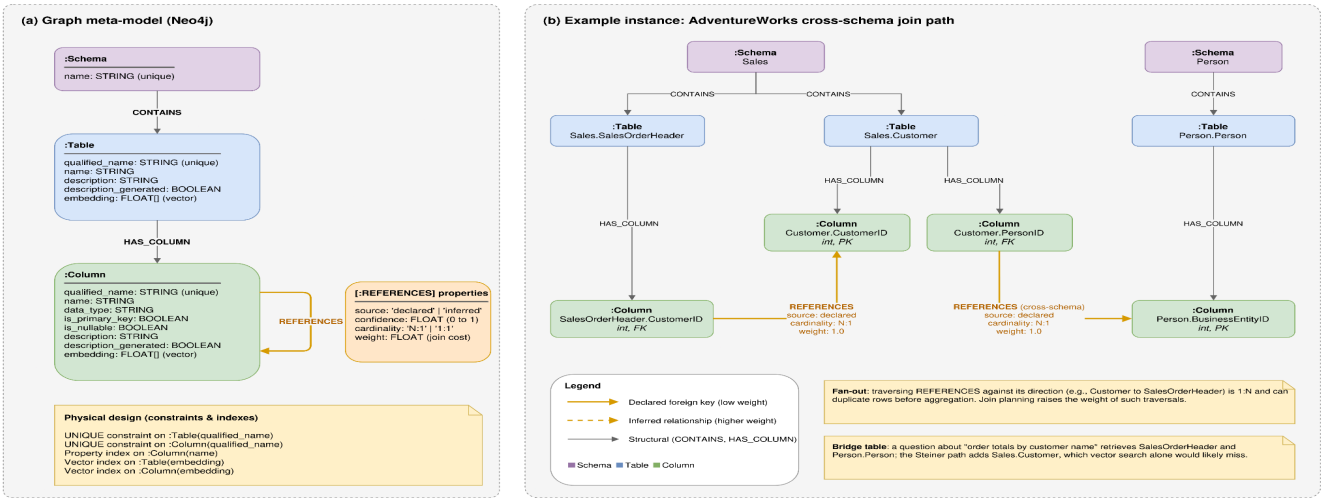


Figure 1: Metadata as a property graph

Modeling foreign keys at the column level means the exact join condition can be read directly from the graph. Physical design includes: uniqueness constraints on fully qualified table and column names, property indices for lookup, and vector indexing for semantic search.

3. Semantic Enrichment & Data Quality

Table and column descriptions are converted into embeddings and stored as a Neo4j vector index. If descriptions are not present, short ones will be generated by an LLM from column names and sampled values. Both generated descriptions and inferred relationships are flagged as lower-quality metadata. This allows the evaluation to assess their impact and exclude them in separate, controlled tests. Handling incomplete and uncertain metadata is a central data quality concern of the project.

4. Graph-guided Schema Linking

For each user question, the following steps will be taken:

1. A vector search identifies the tables most relevant to the question.
2. Join path resolution: the retrieved tables are connected through the graph. For two tables this is a shortest-path problem, for three or more it is the Steiner tree problem, which is solved with an approximation algorithm.
3. Edge weighting: each join is assigned a cost. Inferred relationships receive a higher cost than declared foreign keys so that declared relationships are preferred, and one-to-many joins receive a higher cost because they can duplicate rows in aggregate queries.

4. Including intermediate tables in the subschema

The resulting subschema is specified using DDL and fed as context into the LLM during the query generation pipeline.

5. Query Generation, Validation, & Explanation

The subschema and additional context is used by an LLM to generate SQL code. This is orchestrated by LangChain. Before execution, each generated query is checked with PostgreSQL's *EXPLAIN* command and run under a read-only database role. If a query fails, the database error is returned to the LLM so it can correct the query, up to a fixed number of attempts.

As a result the user receives:

- The result
- The SQL code
- The tables used and why they were selected
- The join path in plain language
- A short explanation of what the query does

A minimal web interface will be built using FastAPI and React to provide users with a window to interact with our pipeline.

Data and Resources:

- Platforms, software, & frameworks:
    - PostgreSQL
    - Python
    - LangChain
    - Neo4j
    - FastAPI
    - React
- Datasets
    - BIRD dataset
    - Adventure Works dataset

AdventureWorks is a Microsoft sample business database with several schemas (Sales, Production, Person, HumanResources, Purchasing) and realistic cross-schema relationships, resembling a database a new employee would face. BIRD is a text-to-SQL benchmark that provides realistic databases, human-written questions with reference SQL, and column descriptions used for semantic enrichment.

Database management component:

We are using a hybrid approach that combines two types of databases: PostgreSQL (relational) and Neo4j (graph).

The first component is PostgreSQL, which holds the AdventureWorks and BIRD datasets. We will extract the metadata (schemas, tables, columns and their data types, primary and foreign keys, unique constraints, and column comments) from its system views. PostgreSQL is also

where the LLM-generated SQL is checked and run, using a read-only role so the queries can't change any data.

The second component is Neo4j. We load the extracted metadata into it as a graph, which gives us a model of how the tables in PostgreSQL actually connect. For each question, Neo4j is used to pick out the small set of tables needed to answer it and figure out how they should be joined.

The pipeline works like this. A user types a question into the React website, which sends it to the FastAPI backend. The backend searches Neo4j for the relevant tables and join paths. LangChain combines the question with those tables to build a prompt, the LLM writes the SQL, and the SQL is run against PostgreSQL.

Evaluation and Expected Outcomes:

The system will be compared against two baselines that differ only in how schema context is selected: the full schema, and retrieval-only vector search without graph expansion. The retrieval-only baseline isolates the graph's contribution. Testing will use about 50 group-written AdventureWorks questions with reference SQL, ranging from single-table queries to cross-schema joins of three or more tables, plus a subset of BIRD's development set. The primary measure is execution accuracy, meaning whether a generated query returns the same results as the reference query. Secondary measures are schema recall, bridge-table recovery, input tokens, latency, and correction attempts. We expect the graph-guided system to match retrieval-only on simple questions, outperform it on multi-table and cross-schema questions, and use far fewer tokens than the full-schema baseline.

Group Responsibilities:
Davin (Team Organizer, Frontend): Builds the frontend website where users enter questions in plain language, leads quality assurance, and assists with the backend. Writes 4.5 pages of the report and presents the introduction and frontend.

Cameron (Graph): Models the database metadata in Neo4j, develops the semantic enrichment and data quality logic, and writes the graph queries that select the best sub-schema for each question. Writes 4.5 pages of the report and presents the graph components.

Emmanuel (Database): Sets up PostgreSQL and loads the BIRD and AdventureWorks datasets. If no suitable library is available, builds the Python ingestion program that extracts the metadata, and builds the evaluation harness that runs generated queries and compares their results against the reference SQL. Writes 4.5 pages of the report and presents the database components .

Mohammed (Middleware): Builds the LangChain pipeline that combines the Neo4j sub-schemas with the LLM prompts for SQL generation, and develops the FastAPI backend that connects all parts of the system . Writes 4.5 pages of the report and presents the middleware components.